Image Releases, Flash-Time Identity and Reproducibility

How IDEA should publish, personalise and check its device images

Proposal · Status: Backlog, awaiting Koen's decision

IDEA - offline school computing. Prepared for Koen Swings's team.
No real passwords, keys or tokens are included; values in <ANGLE BRACKETS> are placeholders.

Contents

1. Background and fundamentals 3
What an image is 3
What "reproducible" means 3
Content-identical, not byte-identical (Koen, 2026-10-26) 3
The manifest (#341) 3
The compare tool 4
The GitHub workflow Koen has to copy himself 4
341 includes a GitHub Actions workflow that runs the repro tools' self-tests on every PR. Bots aren't allowed to push files into .github/workflows/, so it was left at tools/repro/ci/repro-tools.yml. Koen must copy it to .github/workflows/repro-tools.yml himself before it runs 4
2. Option A: publish full images as releases 4
3. Option B: a generic image plus an identity file at flash time 5
4. How the checks fit together 5
5. What the reproducibility check is actually for 6
6. Recommendation (Steve) 6
7. State of the paused #322 work (Atlas, 27 Oct, around 19:20) 6
8. Open questions for Koen 7

Author: Steve (Lead Bot) **Date:** 2026-10-27 **Status:** Backlog, awaiting Koen's decision **Related:** idea#322 (reproducible images), idea#341 (stage 1, merged as `d927e39`), idea#319 and idea#208 (time zone set at flash time, the pattern Option B extends)

This proposal records the discussion of 27 October 2026 so it can be picked up again later without anything being lost. Nothing in it has been decided yet. Section 8 lists the questions for Koen.

1. Background and fundamentals

What an image is

An image is a complete copy of a disk or SD card: the operating system, the installed programs, the IDEA software and all settings, packed into one file. Writing ("flashing") it onto a card or SSD gives a device that starts up ready to use. IDEA has three kinds: the **Engine** image (the school server Pi), the **client** image (the classroom Pi 400s) and the **router** image (the OpenWrt Cudy router).

What "reproducible" means

Every image is built by a recipe kept in git (build scripts plus a list of what goes in). An image is reproducible when someone else can take that recipe at the same git commit, run it on a clean computer and get the same image. That proves the image contains only what the recipe says, with nothing added by hand and nothing hidden.

Content-identical, not byte-identical (Koen, 2026-10-26)

There are two levels of "same":

- **Byte-identical:** the two image files match bit for bit. That is very hard to achieve for a full Linux system: timestamps, the order files are written in, file-system layout and random IDs all differ between builds, even when nothing that matters is different.
- **Content-identical:** every file has the same path, permissions, owner, size and contents. Only things that are bound to differ (timestamps, logs, caches) are allowed to differ.

Koen chose **content-identical** on 2026-10-26. It answers the real question ("is the same software and configuration inside?") at a small fraction of the effort.

The manifest (#341)

Stage 1 (#341, `d927e39`) added a tool that writes a **manifest** of an image: a text file with **one line per file**, giving its path, mode (permissions), owner, size and sha256 fingerprint of the contents.

- The client image s16 gives about **175,000 lines**.
- **Volatile files** (logs, caches, random seeds and the like) are listed but **not compared**, because they always change.

- **Secrets** (keys, passwords) appear only as **fingerprints**, never as their contents, so a manifest can be shared safely.

The compare tool

A second tool compares two manifests and lists every file that was added, removed or changed. As a test it compared client image s15 with s16 and found **exactly 14 differences**: the `config.txt` change from #304, plus the package updates to zlib, protobuf and minizip that came in from Ubuntu between the two builds. Nothing unexpected, which also shows that the tool finds real changes.

The GitHub workflow Koen has to copy himself

341 includes a GitHub Actions workflow that runs the repro tools' self-tests on every PR. Bots aren't allowed to push files into `.github/workflows/`, so it was left at `tools/repro/ci/repro-tools.yml`. Koen must copy it to `.github/workflows/repro-tools.yml` himself before it runs.

2. Option A: publish full images as releases

Each tested image is published as a GitHub Release, and the field flashes exactly that file.

Pros

- The field gets **exactly the image that was tested**, not a rebuild that might differ.
- **Fast restore:** download and flash, with no build step.
- **Independent of upstream changes:** if Ubuntu or OpenWrt change or remove packages, the released image is still there.
- The release **becomes the reference** that later images and field devices are compared with.

Cons and watch points

- **Per-device identity must be removed from the image**, or every copy is a clone: sticker name, Engine ID, Automerge store, SSH host keys, `/etc/machine-id`. Today's duplicate idea-pc-5b4c (two Pis, serials ending 5b4c and 35f2, sharing one name because one card was copied) shows what happens otherwise.
- **A Tailscale fleet key must never be in a public image.** The `<account>911!` passwords and the Wi-Fi keys are different: Koen ruled those non-secret (see `docs/ops/security-overview.md`).
- **GitHub's 2 GB per-file limit** for release files means an Engine image has to be split into parts, or made leaner.
- **Images go stale** as security updates come out, so they need rebuilding regularly.

- **Binaries are hard to review:** a reviewer can't read an image the way they read a script. The manifest and compare tool make up for some of this.
- **The router is the easy case:** the image is small and holds nothing secret, except a Tailscale key if one is ever added.

3. Option B: a generic image plus an identity file at flash time

There is **one generic image per type** (Engine, client, router), with no device identity in it at all.

- The **flash tool** writes the generic image, then puts an **identity file** on the boot partition. This extends what `idea-flash-card` already does today when it patches in the time zone and `--home` (#208/#319).
- At **first boot** the device applies the identity file and deletes it. If there's **no file, it creates a fresh identity**, as `idea-firstboot` already does for clients.

Two uses

1. **A new device:** flash the generic image and give it a new identity.
2. **An exact copy of a field device** from its saved identity, to diagnose a problem at home or to replace a broken device. An Engine copy doesn't need the Engine's data, because **App Disks hold the apps and files**, and the **shared Automerge store fills back in from its peers**.

Watch points

- **Identities stay private**: SSH host keys and any Tailscale key are kept on the bot computer or in Koen's admin tool, **never on GitHub**.
- **Never run a copy alongside the original.** The tool should warn about this and record which identity was used, when and for which card.
- **A replacement shows up as a new Tailscale node**, so the old node must be removed in the Tailscale admin.

The result: the admin tool needs only **a git commit plus an identity file** to describe any device in the field exactly.

4. How the checks fit together

1. **Build** the generic image from git.
2. **Reproducibility check** (once per release): rebuild from git on a clean computer and compare its manifest with the published generic image's manifest.
3. **Flash and patch** per device (identity file, time zone, home mode).
4. **Optional per-device check:** compare the card's manifest with the generic one. Only the identity files should differ.

The **patch step doesn't itself need to be reproducible**: it is small, scripted and checked by step 4. Option B makes the reproducibility check cheaper, because there are **only three generic images to prove**, rather than one per device.

5. What the reproducibility check is actually for

Koen asked what the check is worth. The only use he saw was "reproduce a field setup at home", and stored images plus identity (Options A and B) cover that better. The check has four uses:

1. **Fixing a field device without surprise changes.** When you rebuild to fix one thing, you want only that thing to change, not package drift like the zlib/protobuf/minizip updates between s15 and s16.
2. **Enforcing Koen's rule** that fixes go through the build procedure and images are never edited by hand.
3. **Building again in a year**, when Ubuntu and OpenWrt have moved on.
4. **Handing the project over** to someone who must be able to rebuild everything.

Uses **1 and 2 need only the per-image manifest comparison against the previous release**. That is cheap and already merged in #341. Uses **3 and 4 need the full clean rebuild**, which is the expensive part.

6. Recommendation (Steve)

- **Combine Options A and B:** publish generic images as public releases, and add identity at flash time.
- Make the **manifest comparison against the previous release a standard step for every new image**.
- **Drop the full clean-rebuild proof, or run it once a year**, for example before a handover.

7. State of the paused #322 work (Atlas, 27 Oct, around 19:20)

- **Stage 1** (manifest and compare tools): **merged** in #341 (`d927e39`).
- **Stage 2** (clean rebuild of client commit `13f6917`): **paused** while its inputs were being checked. **Nothing was built or pushed.**
- Kept on Atlas's computer: `/workspace/idea322/s2/s16-pins.tsv` (1,815 pinned packages), the stage-1 evidence in `/workspace/idea322/tmp/`, `APPROACH.md`, and an empty branch `feat/repro-client-rebuild-322` (worktree `/workspace/idea322-s2`).
- Findings:
  - `13f6917`'s `build.sh` doesn't pin package versions, so a rebuild needs a pin step plus access to the Ubuntu mirror or a Launchpad archive (about 2 GB). The box couldn't reach `ports.ubuntu.com`.
  - s16's final shrink to 14 GB was done by hand, which is outside the recipe.

    - s16's `MANIFEST.txt` package list is missing `gdb`, `screen` and `ubuntu-release-upgrader-core`.
- Estimate to resume: **1.5 to 3 hours**, with about **60% confidence of 0 differences**.
- **Stage 3** (router `b2f00bc`), **stage 4** (link deployment records to manifests) and **stage 5** (Engine): **not started**.

8. Open questions for Koen

1. **Option A, Option B, or both?**
2. **Keep the clean rebuild:** never, yearly, or now (resume stage 2)?
3. **Where identity files live:** on the bot computer, or in the admin tool?
4. **Engine images:** split them to fit GitHub's 2 GB limit, or make a leaner image?
5. **Tailscale key:** add it at flash time (through the private identity file), or not at all?